

PHP • FRAMEWORK • WEB & API

Framework

Laravel 12

Do conceito de framework até a construção de uma aplicação web e API REST completas — com banco de dados, autenticação, CRUD e interfaces com Bootstrap.



Projeto prático: Sistema de Cadastro de Equipamentos Industriais

Trilha sequencial: cada tópico assume que o anterior já foi compreendido.

O Caminho que Vamos Percorrer

Cada bloco depende do domínio do bloco anterior — siga a ordem.

1

Fundamentos

Framework, Laravel, pré-requisitos e ambiente de desenvolvimento

2

Configuração

Instalação, estrutura de pastas, .env e conexão com o banco de dados

3

Banco & Ferramentas

SQL (criação/alteração de tabelas), Artisan e Models (Eloquent)

O Caminho que Vamos Percorrer

Continuação: da lógica da aplicação ao resultado final.

4

Lógica da App

Rotas, Controllers e Resource
Controllers

5

Interface

Blade, layouts, componentes e
Bootstrap

6

Prática & Segurança

CRUD completo, validação,
autenticação e API REST



Resultado final: Sistema de Cadastro de Equipamentos com app web + API REST, CRUD e autenticação.

O que é um Framework?

Framework é um conjunto de bibliotecas, regras e uma estrutura de pastas pré-definida que resolve problemas recorrentes do desenvolvimento. **Analogia:** *construir sem framework é erguer uma casa fabricando cada tijolo; com framework, a fundação já vem pronta.*



Estrutura pronta

Pastas e responsabilidades definidas.



Segurança embutida

Proteção contra SQLi, CSRF e XSS.



Produtividade

Menos código repetitivo.

O que é Laravel?

O framework PHP mais usado do mundo para aplicações web modernas.

Laravel é um framework PHP de código aberto que segue o padrão **MVC** (Model-View-Controller), com sintaxe elegante e um ecossistema completo.



Eloquent ORM



Blade Templates



Artisan CLI



Roteamento simples



Auth pronta (Breeze)



Ecossistema (Sanctum...)

Por que Laravel 12?

O que muda entre programar em PHP puro e usar o framework.



PHP Puro

Tudo manual: rotas, conexão SQL, validação e segurança escritos do zero.



Laravel 12

Estrutura, ORM, rotas e segurança prontos — você foca na regra de negócio.



Requisito

PHP ^8.2 instalado + Composer configurado na máquina.

Pré-requisitos e Ambiente

Só avance para a instalação depois de confirmar cada item.



PHP 8.2+

Laravel 12 exige PHP na versão 8.2 ou superior instalado na máquina.



Composer

Gerenciador de dependências PHP. Instala o Laravel e todas as bibliotecas.



Banco de Dados

MySQL, PostgreSQL ou SQLite (ótimo para estudar, sem servidor).

Pré-requisitos e Ambiente

Ferramentas de apoio para escrever, testar e rodar o código.



Editor de Código

VS Code, PhpStorm ou similar,
com terminal integrado.



Node.js + NPM

Necessário mais adiante para
compilar CSS/JS (Vite) com
Bootstrap.



Terminal / CLI

Todo o fluxo Laravel passa por
comandos via linha de
comando.

Instalando o Laravel 12

Com PHP e Composer prontos, criamos o projeto em poucos comandos.

1 Instale o instalador global do Laravel (uma vez por máquina)

```
C:\projeto> composer global require laravel/installer
```

2 Crie o projeto novo

```
C:\projeto> laravel new meu-projeto
```

Rodando o Servidor Local

Depois de criado, o projeto já pode ser executado no navegador.

3 Entre na pasta do projeto e suba o servidor local

```
C:\projeto> cd meu-projeto  
C:\projeto> php artisan serve
```



Acesse **http://localhost:8000** no navegador para ver a tela inicial do Laravel.

Estrutura de Pastas do Projeto

Antes de programar, é preciso saber onde cada peça do Laravel mora.



app/

Models, Controllers e a lógica principal da aplicação.



routes/

Arquivos web.php e api.php — definem as URLs da aplicação.



resources/

Views Blade, CSS e JavaScript (o que o usuário vê).



database/

Scripts SQL, seeders e factories do banco de dados.

Estrutura de Pastas do Projeto

Continuação: configuração, dados sensíveis e arquivos públicos.



config/

Arquivos de configuração (database.php, app.php etc.).



.env

Variáveis de ambiente: credenciais e configs sensíveis.



public/

Ponto de entrada da aplicação (index.php) e assets.



storage/

Logs, cache, uploads e arquivos gerados pela app.

Arquivo .env — Variáveis de Ambiente

Configurações que mudam entre seu PC, o servidor de testes e a produção.

```
.env  
  
APP_NAME="Cadastro de Equipamentos"  
APP_ENV=local  
APP_KEY=base64:XXXXXXXXXXXX  
APP_DEBUG=true  
  
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=equipamentos_industriais  
DB_USERNAME=root  
DB_PASSWORD=
```

- APP_KEY é gerada com php artisan key:generate — nunca deixe em branco.
- APP_DEBUG=true só em desenvolvimento.
- O .env nunca deve ser versionado no Git.
- As variáveis DB_* alimentam config/database.php no próximo passo.

Conectando ao Banco de Dados

O `.env` informa as credenciais; `config/database.php` decide como usá-las.

`config/database.php`

```
'mysql' => [  
    'host'      => env('DB_HOST'),  
    'database'  => env('DB_DATABASE'),  
    'username'  => env('DB_USERNAME'),  
    'password'  => env('DB_PASSWORD'),  
],
```



Testar a conexão: rode `php artisan tinker` e execute `DB::connection()->getPdo()` — se retornar um objeto PDO, a conexão está funcionando.

Artisan CLI — o Canivete Suíço do Laravel

Um comando de terminal para gerar código, rodar tarefas e inspecionar a aplicação.

```
C:\projeto> php artisan list
```

Comando	O que faz
<code>php artisan make:model</code>	Cria um novo Model
<code>php artisan make:controller</code>	Cria um novo Controller
<code>php artisan make:request</code>	Cria uma Form Request
<code>php artisan make:resource</code>	Cria um API Resource (JSON)

Artisan CLI — Mais Comandos do Dia a Dia

Comandos para inspecionar rotas, testar código e subir o servidor.

Comando	O que faz
<code>php artisan route:list</code>	Lista todas as rotas registradas
<code>php artisan tinker</code>	Abre um console PHP interativo
<code>php artisan db:show</code>	Mostra as tabelas do banco conectado
<code>php artisan serve</code>	Sobe o servidor local de desenvolvimento

Criando o Banco de Dados com SQL

Sem migrations neste curso: as tabelas nascem direto em SQL puro.

SQL – tabela setores

```
CREATE TABLE setores (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);
```

SQL – populando com dados iniciais

```
INSERT INTO setores (nome) VALUES  
  ('Manutenção'), ('Produção');
```

Criando a Tabela de Equipamentos

A tabela principal do projeto, com chave estrangeira para setores.

SQL – tabela equipamentos

```
CREATE TABLE equipamentos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(150) NOT NULL,  
  patrimonio VARCHAR(30) NOT NULL UNIQUE,  
  setor_id INT,  
  status VARCHAR(20) DEFAULT 'ativo',  
  FOREIGN KEY (setor_id) REFERENCES setores(id)  
);
```

Alterando Tabelas com ALTER TABLE

O banco muda junto com o projeto.

Adicionar uma coluna

SQL

```
ALTER TABLE equipamentos ADD COLUMN localizacao VARCHAR(100);
```

Modificar o tipo de uma coluna

SQL

```
ALTER TABLE equipamentos MODIFY COLUMN nome VARCHAR(200) NOT NULL;
```

Removendo Colunas

Cuidado: remover uma coluna apaga os dados que estavam nela.

Remover uma coluna

SQL

```
ALTER TABLE equipamentos DROP COLUMN localizacao;
```



Depois de alterar a tabela no banco, atualize também o **\$fillable** do Model correspondente.

Blade — Exibindo Dados e Condicionais

Antes do CRUD, o essencial de Blade: HTML puro + diretivas simples.

```
resources/views/equipamentos/index.blade.php

<h1>Equipamentos</h1>

@if ($equipamentos->isEmpty())
    <p>Nenhum equipamento cadastrado.</p>
@else
    @foreach ($equipamentos as $eq)
        <h2>{{ $eq->nome }}</h2>
    @endforeach
@endif
```

Blade — Formulário e Proteção CSRF

Todo formulário POST/PUT/DELETE no Laravel exige o token @csrf.

```
resources/views/equipamentos/create.blade.php

<form method="POST" action="{{ route('equipamentos.store') }}">
    @csrf
    <!-- campos do formulário -->
</form>
```

 **@csrf** gera um token oculto que impede que sites maliciosos enviem formulários em nome do usuário logado.

Blade — Criando um Layout Base

Evite repetir HTML: um layout-base é herdado por todas as páginas.

resources/views/layouts/app.blade.php

```
<html>
<head>
  <title>@yield('title')</title>
</head>
<body>
  @include('partials.nav')
  <main>
    @yield('content')
  </main>
</body>
</html>
```

Blade — Herdando o Layout

Cada página só precisa declarar o que muda: título e conteúdo.

resources/views/equipamentos/index.blade.php

```
@extends('layouts.app')

@section('title', 'Equipamentos')

@section('content')
    <h1>Equipamentos</h1>
@endsection
```

@extends herda o layout • **@yield** / **@section** definem os blocos de conteúdo

Integrando Bootstrap nas Views

Um CDN no layout já libera classes prontas de estilo e componentes.

layouts/app.blade.php – dentro de <head>

```
<link href=".../bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
<script src=".../bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

- CSS no <head>: estiliza botões, formulários e grid.
- JS no fim do <body>: ativa componentes interativos (menus, modais).
- As classes Bootstrap (btn, form-control, mb-3...) já ficam disponíveis em qualquer view Blade.

Um Formulário com Bootstrap

```
resources/views/equipamentos/create.blade.php

<form method="POST" action="{ route('equipamentos.store') }"
      class="container mt-4">
  @csrf
  <div class="mb-3">
    <label class="form-label">Nome</label>
    <input name="nome" class="form-control">
  </div>
  <div class="mb-3">
    <label class="form-label">Setor</label>
    <select name="setor_id" class="form-select">...</select>
  </div>
  <button class="btn btn-danger">Salvar</button>
</form>
```

Model — Setor

O primeiro Model do projeto: simples, sem dependências.

```
C:\projeto> php artisan make:model Setor
```

app/Models/Setor.php

```
class Setor extends Model
{
    protected $fillable = ['nome'];
}
```

Rotas — Setor

A rota é a porta de entrada: define qual URL aciona qual código.

routes/web.php

```
use App\Http\Controllers\SetorController;

Route::get('/setores',
    [SetorController::class, 'index'])
    ->name('setores.index');
```

Resource Controller — Setor

Um único comando gera as 7 ações padrão de um CRUD.

```
C:\projeto> php artisan make:controller SetorController --resource
```

routes/web.php

```
Route::resource('setores', SetorController::class);
```

As 7 Ações do CRUD

Cada verbo HTTP + URI já aciona o método certo no Controller.

Verbo	URI	Método	Ação
GET	/setores	index	Listar todos
GET	/setores/create	create	Form de criação
POST	/setores	store	Salvar novo
GET	/setores/{id}	show	Exibir um
GET	/setores/{id}/edit	edit	Form de edição
PUT	/setores/{id}	update	Atualizar
DELETE	/setores/{id}	destroy	Remover

Controller — Método index()

```
C:\projeto> php artisan make:controller SetorController
```

```
app/Http/Controllers/SetorController.php

class SetorController extends Controller
{
    public function index()
    {
        $setores = Setor::all();
        return view('setores.index', compact('setores'));
    }
}
```

CRUD de Setor — Index (Tabela)

Listando os setores em formato de tabela, com uma coluna de Ações.

resources/views/setores/index.blade.php

```
<table class="table">
  <tr><th>Nome</th><th>Ações</th></tr>
  @foreach ($setores as $setor)
    <tr>
      <td>{{ $setor->nome }}</td>
      <td><!-- editar / excluir --></td>
    </tr>
  @endforeach
</table>
```


CRUD de Setor — Coluna de Ações

Editar leva para a tela de edição; Excluir remove o registro na hora.

```
resources/views/setores/index.blade.php – dentro do <td>

<a href="{{ route('setores.edit', $setor->id) }}"
  class="btn btn-primary btn-sm">Editar</a>

<form method="POST" class="d-inline"
  action="{{ route('setores.destroy', $setor->id) }}">
  @csrf @method('DELETE')
  <button class="btn btn-danger btn-sm">Excluir</button>
</form>
```

CRUD de Setor — Create

O setor precisa existir antes de qualquer equipamento poder referenciá-lo.

app/Http/Controllers/SetorController.php

```
public function store(Request $request)
{
    Setor::create($request->only('nome'));
    return redirect()->route('setores.index');
}
```

CRUD de Setor — View Create

O formulário que o técnico preenche para cadastrar um novo setor.

resources/views/setores/create.blade.php

```
<form method="POST" action="{{ route('setores.store') }}"
      class="container mt-4">
  @csrf
  <div class="mb-3">
    <label class="form-label">Nome do Setor</label>
    <input name="nome" class="form-control">
  </div>
  <button class="btn btn-danger">Salvar</button>
</form>
```

CRUD de Setor — Update & Delete

app/Http/Controllers/SetorController.php

```
public function update(Request $request, string $id)
{
    $setor = Setor::find($id);
    $setor->update($request->only('nome'));
    return redirect()->route('setores.index');
}

public function destroy(string $id)
{
    $setor = Setor::find($id);
    $setor->delete();
    return redirect()->route('setores.index');
}
```

CRUD de Setor — View Edit

Mesmo formulário do Create, mas pré-preenchido e com @method('PUT').

```
resources/views/setores/edit.blade.php

<form method="POST"
      action="{{ route('setores.update', $setor->id) }}"
      class="container mt-4">
  @csrf @method('PUT')
  <div class="mb-3">
    <label class="form-label">Nome do Setor</label>
    <input name="nome" class="form-control"
          value="{{ $setor->nome }}">
  </div>
  <button class="btn btn-primary">Atualizar</button>
</form>
```

Validação — Setor

Nunca confie no que o usuário envia — valide antes de salvar.

SetorController.php

```
$request->validate([  
    'nome' => 'required|max:100|unique:setores',  
]);
```

✓ **CRUD de Setor completo!** Agora o mesmo caminho será feito para Equipamento — um pouco mais completo.

Ação Personalizada — Campo Ativo

Antes de ativar/desativar setores, o banco precisa guardar esse status.

SQL

```
ALTER TABLE setores ADD COLUMN ativo BOOLEAN DEFAULT TRUE;
```

Rota Personalizada — ativar-desativar

Além das 7 ações padrão, você pode registrar rotas próprias no Controller.

routes/web.php

```
Route::patch('/setores/{id}/status',  
    [SetorController::class, 'ativarDesativar'])  
    ->name('setores.ativar-desativar');
```


Controller — Método ativarDesativar()

Inverte o valor de ativo e salva — uma ação personalizada, fora do CRUD padrão.

app/Http/Controllers/SetorController.php

```
public function ativarDesativar(string $id)
{
    $setor = Setor::find($id);
    $setor->ativo = !$setor->ativo;
    $setor->save();
    return redirect()->route('setores.index');
}
```

Botão Ativar/Desativar na Listagem

Um botão na coluna Ações dispara a rota personalizada.

resources/views/setores/index.blade.php – dentro do <td>

```
<form method="POST"
      action="{{ route('setores.ativar-desativar', $setor->id) }}">
  @csrf
  @method('PATCH')
  <button class="btn btn-sm
    {{ $setor->ativo ? 'btn-warning' : 'btn-success' }}">
    {{ $setor->ativo ? 'Desativar' : 'Ativar' }}
  </button>
</form>
```

Label de Status — Ativo/Desativado

Um badge colorido deixa o status visível de imediato na tabela.

resources/views/setores/index.blade.php – dentro do <td>

```
@if ($setor->ativo)
    <span class="badge bg-success">Ativo</span>
@else
    <span class="badge bg-danger">Desativado</span>
@endif
```

Model — Equipamento

Com a tabela já criada via SQL, o Model é a classe PHP que conversa com ela.

```
C:\projeto> php artisan make:model Equipamento
```

```
app/Models/Equipamento.php

class Equipamento extends Model
{
    protected $fillable = ['nome', 'patrimonio',
                           'setor_id', 'status'];
}
```

Consultas Eloquent Mais Usadas

O mesmo padrão vale para qualquer Model — aqui com Equipamento.

PHP (Tinker ou Controller)

```
Equipamento::all();           // todos os equipamentos
Equipamento::find(1);         // busca por id

Equipamento::create([
    'nome' => 'Torno CNC', 'patrimonio' => 'PAT-0021',
]);

$eq = Equipamento::find(1);
$eq->update(['status' => 'manutencao']);
$eq->delete();
```

Relacionamentos — Setor

Um setor tem vários equipamentos: hasMany.

app/Models/Setor.php

```
class Setor extends Model
{
    public function equipamentos()
    {
        return $this->hasMany(Equipamento::class);
    }
}
```

Relacionamentos — Equipamento e Uso

Um equipamento pertence a um setor: belongsTo.

app/Models/Equipamento.php

```
class Equipamento extends Model
{
    public function setor()
    {
        return $this->belongsTo(Setor::class);
    }
}
```

`$setor->equipamentos` e **`$equipamento->setor`** acessam o relacionamento.

Rotas — Equipamento

Mesma lógica das rotas de Setor, agora para o recurso principal.

routes/web.php

```
use App\Http\Controllers\EquipamentoController;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/equipamentos',
    [EquipamentoController::class, 'index'])
    ->name('equipamentos.index');
```


Resource Controller — Equipamento

O mesmo comando, aplicado ao recurso principal do sistema.

```
C:\projeto> php artisan make:controller EquipamentoController --resource
```

routes/web.php

```
Route::resource('equipamentos', EquipamentoController::class);
```

Controller — Método index()

```
C:\projeto> php artisan make:controller EquipamentoController
```

```
app/Http/Controllers/EquipamentoController.php

class EquipamentoController extends Controller
{
    public function index()
    {
        $equipamentos = Equipamento::all();
        return view('equipamentos.index',
                    compact('equipamentos'));
    }
}
```

CRUD de Equipamento — Index (Tabela)

Listando os equipamentos em formato de tabela, com uma coluna de Ações.

resources/views/equipamentos/index.blade.php

```
<table class="table">
  <tr>
    <th>Nome</th><th>Patrimônio</th><th>Status</th><th>Ações</th>
  </tr>
  @foreach ($equipamentos as $eq)
    <tr>
      <td>{{ $eq->nome }}</td>
      <td>{{ $eq->patrimonio }}</td>
      <td>{{ $eq->status }}</td>
      <td><!-- editar / excluir --></td>
    </tr>
  @endforeach
</table>
```

CRUD de Equipamento — Coluna de Ações

Editar leva para a tela de edição; Excluir remove o registro na hora.

```
resources/views/equipamentos/index.blade.php – dentro do <td>

<a href="{{ route('equipamentos.edit', $eq->id) }}"
  class="btn btn-primary btn-sm">Editar</a>

<form method="POST" class="d-inline"
  action="{{ route('equipamentos.destroy', $eq->id) }}">
  @csrf @method('DELETE')
  <button class="btn btn-danger btn-sm">Excluir</button>
</form>
```

Controller — Método show()

Sem Route-Model Binding: o id chega como string e buscamos o registro manualmente.

```
app/Http/Controllers/EquipamentoController.php

public function show(string $id)
{
    $equipamento = Equipamento::find($id);
    return view('equipamentos.show',
        compact('equipamento'));
}
```

ID **Sem Route-Model Binding:** recebemos string `$id` e buscamos o registro manualmente com `Equipamento::find($id)`.

CRUD de Equipamento — Create

Do formulário ao banco: o método store() salva o novo registro.

```
app/Http/Controllers/EquipamentoController.php

public function store(Request $request)
{
    $dados = $request->only(
        'nome', 'patrimonio', 'setor_id');
    Equipamento::create($dados);
    return redirect()->route('equipamentos.index');
}
```

CRUD de Equipamento — View Create

resources/views/equipamentos/create.blade.php

```
<form method="POST" action="{{ route('equipamentos.store') }}"
      class="container mt-4">
    @csrf
    <div class="mb-3">
        <label class="form-label">Nome</label>
        <input name="nome" class="form-control">
    </div>
    <div class="mb-3">
        <label class="form-label">Patrimônio</label>
        <input name="patrimonio" class="form-control">
    </div>
    <div class="mb-3">
        <label class="form-label">Setor</label>
        <select name="setor_id" class="form-select">...</select>
    </div>
    <button class="btn btn-danger">Salvar</button>
</form>
```

CRUD de Equipamento — Update & Delete

app/Http/Controllers/EquipamentoController.php

```
public function update(Request $request, string $id)
{
    $equipamento = Equipamento::find($id);
    $equipamento->update($request->only(
        'nome', 'status'));
    return redirect()->route('equipamentos.index');
}

public function destroy(string $id)
{
    $equipamento = Equipamento::find($id);
    $equipamento->delete();
    return redirect()->route('equipamentos.index');
}
```


CRUD de Equipamento — View Edit

```
resources/views/equipamentos/edit.blade.php

<form method="POST"
    action="{{ route('equipamentos.update', $equipamento->id) }}"
    class="container mt-4">
    @csrf @method('PUT')
    <div class="mb-3">
        <label class="form-label">Nome</label>
        <input name="nome" class="form-control" value="{{ $equipamento->nome }}">
    </div>
    <div class="mb-3">
        <label class="form-label">Status</label>
        <select name="status" class="form-select">...</select>
    </div>
    <button class="btn btn-primary">Atualizar</button>
</form>
```

Validação Rápida — Equipamento

O mesmo princípio visto em Setor, com mais campos e regras.

EquipamentoController.php

```
$request->validate([  
    'nome'          => 'required|max:255',  
    'patrimonio'    => 'required|unique:equipamentos',  
    'setor_id'      => 'required|exists:setores,id',  
]);
```

Validação Reutilizável — Form Request

Regras de validação isoladas em sua própria classe.

```
C:\projeto> php artisan make:request StoreEquipamentoRequest
```

```
app/Http/Requests/StoreEquipamentoRequest.php

public function rules(): array
{
    return [
        'nome' => 'required',
    ];
}
```

Validação — Exibindo o Erro na View

Se a validação falhar, o Laravel volta para o formulário com a mensagem.

```
resources/views/equipamentos/create.blade.php

@error('patrimonio')
    <div class="text-danger">{{ $message }}</div>
@enderror
```

Autenticação — Instalando o Breeze

Só técnicos autenticados podem cadastrar ou editar equipamentos.

```
C:\projeto> composer require laravel/breeze --dev
C:\projeto> php artisan breeze:install
C:\projeto> npm install && npm run build
C:\projeto> php artisan migrate
```

Protegendo Rotas com Middleware

O middleware auth barra quem não estiver logado.

routes/web.php

```
Route::middleware('auth')->group(function () {  
    Route::resource('equipamentos', EquipamentoController::class);  
});
```

Diretivas Blade — @auth e @guest

Mostra ou esconde trechos da view conforme o usuário está logado.

```
resources/views/layouts/app.blade.php

@auth
    <span>Olá, {{ Auth::user()->name }}!</span>
@endauth
@guest
    <a href="{{ route('login') }}">Entrar</a>
@endguest
```

 **@auth / @guest** mostram ou escondem o bloco conforme o login — ótimos no menu de navegação.

A Facade Auth — Verificando Login

Métodos para checar o login e obter dados do usuário autenticado.

PHP — Controller ou Blade

```
Auth::check();    // true se houver usuário logado
Auth::user();     // instância do usuário autenticado
Auth::id();       // apenas o id do usuário logado
Auth::attempt(['email' => $email, 'password' => $senha]);
Auth::logout();   // encerra a sessão do usuário
```

 **Auth::attempt()** compara o password enviado com o hash salvo no banco — nunca compare senhas com ==.

Login, Logout e Middleware guest

O Breeze já registra essas rotas; veja o papel de cada middleware.

routes/auth.php (gerado pelo Breeze)

```
Route::get('login', [AuthController::class, 'create'])
    ->middleware('guest')->name('login');
Route::post('login', [AuthController::class, 'store'])
    ->middleware('guest');
Route::post('logout', [AuthController::class, 'destroy'])
    ->middleware('auth')->name('logout');
```



guest vs. auth: guest bloqueia quem já está logado de acessar /login; auth exige sessão ativa para /logout.

API REST — Setor

Mesma base de Models e Controllers — a diferença é responder em JSON.

routes/api.php

```
Route::apiResource('setores', SetorApiController::class);
```

SetorApiController.php

```
public function index()
{
    return Setor::all();
    // Eloquent já responde JSON
}
```

API REST — Equipamento

O recurso principal, seguindo o mesmo padrão.

routes/api.php

```
Route::apiResource('equipamentos', EquipamentoApiController::class);
```

EquipamentoApiController.php

```
public function index()
{
    return Equipamento::all();
    // Eloquent já responde JSON
}
```

Resposta da API em JSON

É assim que um app mobile ou frontend externo consome os dados.

```
GET /api/equipamentos

[
  { "nome": "Torno CNC", "status": "ativo" },
  { "nome": "Prensa" }
]
```

php artisan make:resource EquipamentoResource — formata a saída com mais controle.

Autenticação de API — Sanctum

Sanctum emite tokens para apps mobile e SPAs consumirem a API com segurança.

```
C:\projeto> composer require laravel/sanctum && php artisan install:api
```



```
routes/api.php

Route::middleware('auth:sanctum')
    ->group(function () {
        Route::apiResource('equipamentos',
            EquipamentoApiController::class);
    });
```

Gerando e Usando o Token

O token vai no header Authorization de cada requisição autenticada.

AuthController.php

```
$token = $user->createToken('app')->plainTextToken;  
  
return response()->json(['token' => $token]);
```

cURL / Postman

```
Authorization: Bearer <token>
```

PARA FECHAR O CURSO

Boas Práticas e Próximos Passos

O caminho percorrido: framework → ambiente → SQL → Blade → CRUD de Setor → CRUD de Equipamento → autenticação → API.



.env fora do Git

Nunca versione credenciais.



Sempre valide

validate() ou Form Request.



Escreva testes

php artisan make:test.



Documente a API

Rotas, parâmetros e respostas.



Controllers enxutos

Regras complexas vão para Services.



Próximo passo

Livewire, filas e deploy.